Seguridad y Calidad

Clasificación de servicios

- ***IaaS (Infrastructure as a Service):*** Amazon EC2, Microsoft Azure
- ***PaaS (Platform as a Service):*** Heroku, Google App Engine
- ***SaaS (Software as a Service):*** Google Drive, Facebook, etc

12 Factores (en aplicaciones web)

Objetivos:

- Usar configuraciones automatizables y explícitas, nada manual.
- Dependencias claras con el entorno en el que se ejecutan para máxima compatibilidad.
- Tener en cuenta que la aplicación será desplegada en la nube.
- Despliegue continuo, por tanto requiere coordinación de entornos (test-uat-pre-pro…).
- Crear aplicaciones con la escalabilidad en la mente.

Código Base

El código debe estar en repositorios y es desde ahí donde se harán los despliegues a los entornos. Puede haber varios repositorios si la aplicación es compleja, pero no se puede repetir código entre los repositorios.

Dependencias

Todas las dependencias de la aplicación deben ser explícitas y existir un sistema automático que las instale si no existen en el entorno.

Configuraciones

El código no puede tener configuraciones, sino que deben ser separadas en otros recursos dependientes del entorno. Así, el código debe hacer referencia a las variables de configuración pero no a los valores. Esto permite instalar el mismo código en diferentes entornos.

Baking Service

Son recursos conectables a la aplicación, independientemente de si son locales o de terceros. Si la aplicación necesita una base de datos, será tratada como un recurso independiente, con su URL y su instalación, que deben ser proporcionados a la aplicación.

Construir, Distribuir y Ejecutar

Son las 3 etapas que se deben pasar para transformar el código fuente en un producto utilizable. Deben ser tratadas de forma independiente y con un control de las versiones que se van distribuyendo acorde a un sistema preestablecido.

Proceso sin Estado (stateless)

La aplicación se ejecuta en el destino como un proceso (o varios) en la máquina anfitriona pero la información que existe en su espacio de memoria sólo tiene lo necesario para hacer las transacciones que lleva a cabo en ese momento, no para atender a transacciones futuras: puede reiniciarse la máquina, aparecer otro proceso… La información que sea necesario almacenar para el futuro se hará en un Backing Service.

Asignación de Puerto

El código no puede considerar a priori la asignación de los puertos. Simplemente sabe que es un proceso que escucha en un puerto arbitrario. Será la configuración la que establezca los puertos de cada componente.

Concurrencia

Las aplicaciones deben estar desarrolladas teniendo en mente la concurrencia, bien de hilos o bien de procesos. La razón principal es la naturaleza distribuida y la capacidad para escalar levantando nuevas máquinas

Disponibilidad / Desechabilidad

Las aplicaciones deben ser capaces de iniciar sus procesos rápidamente y de responder de forma controlada a la finalización, aunque esta sea inesperada. Cuando hablamos de escalado, nos referimos al inicio o parada en caliente de nuevas máquinas o al traslado a diferentes máquinas de los procesos.

Partida en el Desarrollo y Producción

Se refiere a tener los diferentes entornos (des-pre como mínimo) lo más similares posibles, tanto de versiones de código desplegado, como de entornos y servicios. Esto se hace con despliegues continuos, lo que implica automatizar los despliegues lo máximo posible, que el personal de desarrollo sea mucho más consciente del entorno de producción y que incluso se compartan modos de trabajo entre entornos aunque no sea lo más evidente a primera vista: puppet, chef, ansible.

Historial

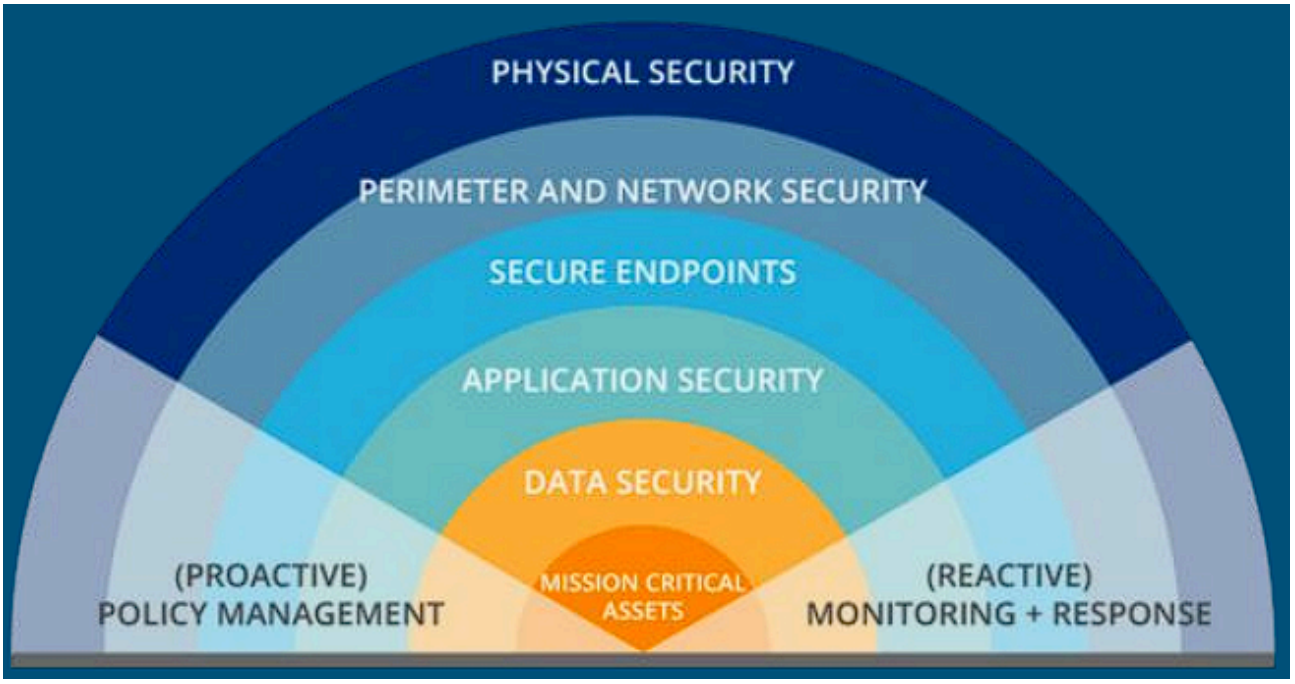
O más conocidos como “Logs”. Las aplicaciones no se preocupan del direccionamiento y almacenamiento de sus datos de logs. Para eso existen sistemas específicos en cada entorno.

Administrador de Procesos

Además de los procesos propios de la ejecución de la aplicación, existirán procesos de tareas de administración y mantenimiento que suelen ejecutarse una única vez (ej: migración de bases de datos, ejecuciones de consola) . Estos procesos, aunque puedan parecer secundarios, se ejecutarán con igual importancia que el resto de los procesos de la aplicación.

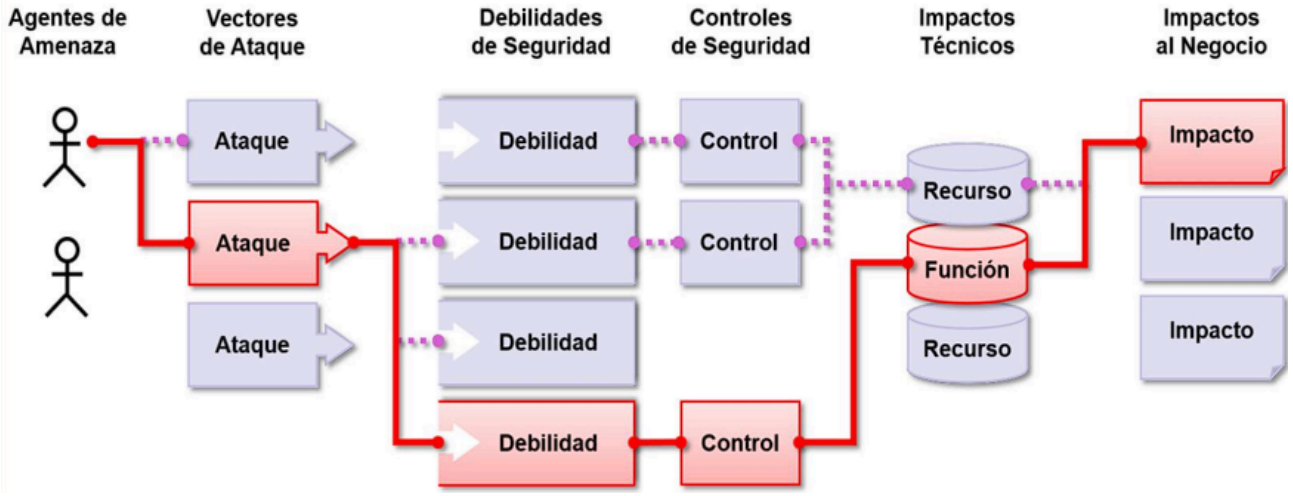
Conclusión: Las nuevas opciones para la distribución de las aplicaciones en Internet afectan a la forma de desarrollarlas y en general a su ciclo de vida: la necesidad de adaptarse a la demanda, tanto en número como nuevas capacidades, y hacerlo con una demora cada vez menor, nos fuerza a adoptar nuevos enfoques.

Seguridad en Aplicaciones



OWASP (*Open Web Application Security Project*) es un proyecto de código abierto dedicado a determinar y combatir las causas que hacen que el software sea inseguro.

Riesgo en seguridad de Aplicaciones



Resumen de el top 10 de riegos en el 2017

Riesgo	Agentes de Amenaza	Vectores de Ataque	Debilidades de Seguridad		Impacto		Puntuación
		Explotabilidad	Prevalencia	Detectabilidad	Técnico	Negocio	
A1: 2017- Inyección	Específico de la Aplicación	FACIL: 3	COMUN: 2	FACIL: 3	GRAVE: 3	Específico de la Aplicación	8,0
A2: 2017 - Pérdida de Autenticación	Específico de la Aplicación	FACIL: 3	COMUN: 2	PROMEDIO: 2	GRAVE: 3	Específico de la Aplicación	7,0
A3: 2017 - Exposición de Datos Sensibles	Específico de la Aplicación	PROMEDIO: 2	DIFUNDIDO: 3	PROMEDIO: 2	GRAVE: 3	Específico de la Aplicación	7,0
A4: 2017 - Entidad Externa de XML (XXE)	Específico de la Aplicación	PROMEDIO: 2	COMUN: 2	FACIL: 3	GRAVE: 3	Específico de la Aplicación	7,0
A5: 2017 - Pérdida de Control de Acceso	Específico de la Aplicación	PROMEDIO: 2	COMUN: 2	PROMEDIO: 2	GRAVE: 3	Específico de la Aplicación	6,0
A6: 2017 - Configuración de Seguridad Incorrecta	Específico de la Aplicación	FACIL: 3	DIFUNDIDO: 3	FACIL: 3	MODERADO: 2	Específico de la Aplicación	6,0
A7: 2017 - Secuencia de Comandos en Sitios Cruzados (XSS)	Específico de la Aplicación	FACIL: 3	DIFUNDIDO: 3	FACIL: 3	MODERADO: 2	Específico de la Aplicación	6,0
A8: 2017 - Deserialización Insegura	Específico de la Aplicación	DIFICIL: 1	COMUN: 2	PROMEDIO: 2	GRAVE: 3	Específico de la Aplicación	5,0
A9: 2017 - Componentes con Vulnerabilidades Conocidas	Específico de la Aplicación	PROMEDIO: 2	DIFUNDIDO: 3	PROMEDIO: 2	MODERADO: 2	Específico de la Aplicación	4,7
A10: 2017 - Registro y Monitoreo Insuficientes	Específico de la Aplicación	PROMEDIO: 2	DIFUNDIDO: 3	DIFICIL: 1	MODERADO: 2	Específico de la Aplicación	4,0

Este cuadro se elaboró con la información recolectada en los proyectos que colaboran. El top se basa en las causas de las vulnerabilidades más comunes. Se basa en la incidencia y no en la frecuencia (la vulnerabilidad se cuenta una vez por informe).

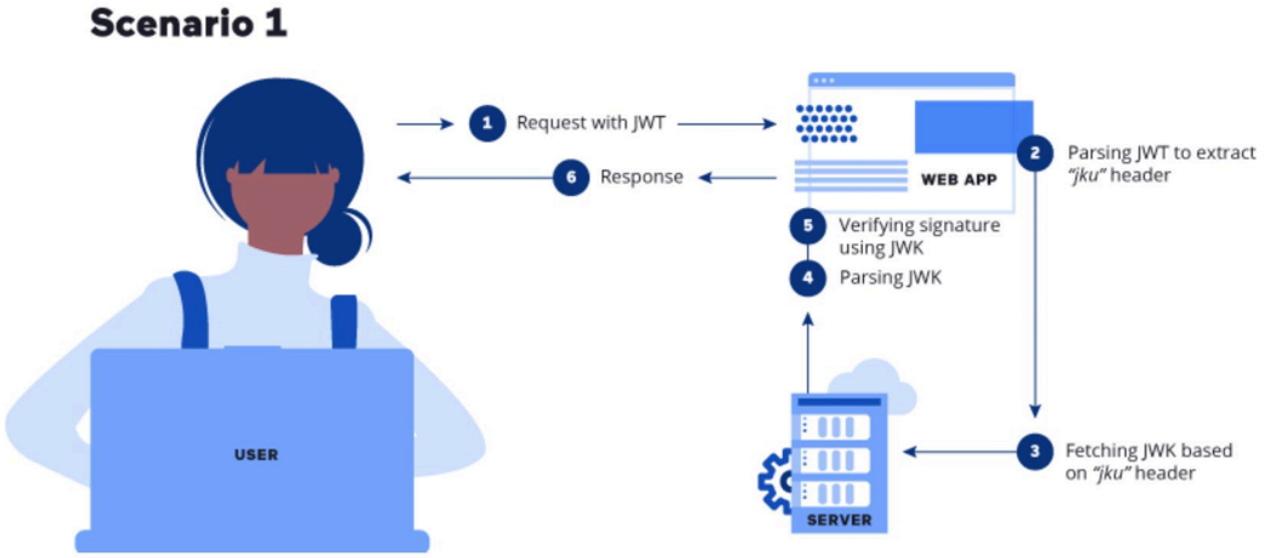
CVE (Common Vulnerabilities and Exposures) es una lista pública de datos sobre las vulnerabilidades comunes que se han reportado de todos los softwares. El sistema CVE permite encontrar todos los fallos de seguridad que se han hecho públicos.

Top 10 Ataques de 2021

A1 - Inadecuado control de acceso

Los controles de acceso limitan a los usuarios a actuar fuera de los permisos previstos. Cuando este proceso falla, la consecuencia suele ser la divulgación de información, la modificación o destrucción de todos los datos o la ejecución de una función de negocio de forma no autorizada

Ejemplos de escenarios de ataque:



A2 - Fallos criptográficos

La criptografía protege de robo y acceso no autorizado a la información sensible y que requiere protección adicional. Algunos motivos son:

- ¿Se transmite algún dato en texto claro? Esto se refiere a protocolos como HTTP, SMTP, FTP que también usan actualizaciones de TLS como STARTTLS. El tráfico de Internet externo es peligroso. Verifique todo el tráfico interno, por ejemplo, entre balanceadores de carga, servidores web o sistemas back-end.
- ¿Se utilizan protocolos o algoritmos criptográficos antiguos o débiles de forma predeterminada o en código antiguo?
- ¿Están en uso las claves criptográficas predeterminadas, se generan o reutilizan claves criptográficas débiles, o falta una gestión o rotación de claves adecuada? ¿Se registran las claves criptográficas en los repositorios de código fuente?
- ¿Se utilizan funciones hash en desuso, como MD5 o SHA1, o se utilizan funciones hash no criptográficas cuando se necesitan funciones hash criptográficas?

Ejemplo de escenario de ataque:

La base de datos de contraseñas utiliza hashes simples o sin sal para almacenar las contraseñas de todos. Una falla en la carga de archivos permite que un atacante recupere la base de datos de contraseñas. Todos los hashes sin sal se pueden exponer con una tabla de arcoíris de hashes precalculados. Los hashes generados por funciones hash simples o rápidas pueden ser descifrados por GPU, incluso si estuvieran salados.

A3 - Inyección

Las fallas de inyección, como SQL, NoSQL, OS o LDAP ocurren cuando se envían datos no confiables a un intérprete, como parte de un comando o consulta. Los datos dañinos del atacante pueden engañar al intérprete para que ejecute comandos involuntarios o acceda a los datos sin la debida autorización.

Una aplicación es vulnerable a ataques de este tipo cuando:

- Los datos suministrados por el usuario no son validados, filtrados o sanitizados por la aplicación.
- Se invocan consultas dinámicas o no parametrizadas, sin codificar los parámetros de forma acorde al contexto.
- Se utilizan datos dañinos dentro de los parámetros de búsqueda en consultas Object-Relational Mapping (ORM), para extraer registros adicionales sensibles.
- Los datos dañinos se usan directamente o se concatenan, de modo que el SQL o comando resultante contiene datos y estructuras con consultas dinámicas, comandos o procedimientos almacenados.

Ejemplos de escenarios de ataque:

1) La aplicación utiliza datos no confiables en la construcción del siguiente comando SQL vulnerable:

```
String query = "SELECT * FROM accounts WHERE custID='" +
request.getParameter("id") + "'";
```

2) La confianza total de una aplicación en su framework puede resultar en consultas que aún son vulnerables a inyección, por ejemplo, Hibernate Query Language (HQL):

```
Query HQLQuery = session.createQuery("FROM accounts WHERE custID='" +
request.getParameter("id") + "')
```

En ambos casos, el atacante puede modificar el parámetro “id” en su navegador para enviar: ' or '1'='1. Por ejemplo: *http://example.com/app/accountView?id=' or '1'='1*

Esto cambia el significado de ambas consultas, devolviendo todos los registros de la tabla “accounts”.

Ataques más peligrosos podrían modificar los datos o incluso invocar procedimientos almacenados.

A4 - Diseño inseguro

Los problemas de diseño provocan la existencia de vulnerabilidades de base que son difíciles de corregir y que requieren volver a escribir gran cantidad de código. No es lo mismo el diseño inseguro a la implementación segura

Formas de evitarlo:

- Utilizar un ciclo de desarrollo seguro: Considere aprovechar el Modelo de Madurez de Software Assurance (SAMM) de OWASP para ayudar a estructurar sus esfuerzos de desarrollo de software seguro.
- Establezca y utilice una biblioteca de patrones de diseño seguros o componentes de caminos pavimentados listos para usar
- Utilice el modelado de amenazas para la autenticación crítica, el control de acceso, la lógica empresarial y los flujos de claves

A5 - Configuración de seguridad incorrecta

Algunos ejemplos de este caso son:

- ➢ Le falta el hardening de seguridad adecuado en cualquier parte del stack tecnológico o permisos configurados incorrectamente en los servicios en la nube.
- ➢ Tiene funciones innecesarias habilitadas o instaladas (puertos, servicios, páginas, cuentas o privilegios innecesarios, por ejemplo).
- ➢ Las cuentas predeterminadas y sus contraseñas aún están habilitadas y sin cambios.
- ➢ El manejo de errores revela a los usuarios rastros de pila u otros mensajes de error demasiado informativos.

A6 - Componentes vulnerables y desactualizados

Probablemente sea vulnerable:

- ➢ Si no conoce las versiones de todos los componentes que utiliza (tanto en el cliente como en el servidor). Esto incluye los componentes que usa directamente, así como las dependencias anidadas.
- ➢ Si el software es vulnerable, carece de soporte o no está actualizado. Esto incluye el sistema operativo, el servidor web/de aplicaciones, el sistema de administración de bases de datos (DBMS), las aplicaciones, las API y todos los componentes, los entornos de ejecución y las bibliotecas.
- ➢ Si no analiza en búsqueda de vulnerabilidades de forma regular y no se suscribe a los boletines de seguridad relacionados con los componentes que utiliza.

A7 - Fallos de Identificación y autenticación

Puede haber debilidades de autenticación si la aplicación:

- ➢ Permite ataques automatizados como la reutilización de credenciales conocidas, donde el atacante posee una lista de pares de usuario y contraseña válidos.
- ➢ Permite ataques de fuerza bruta u otros ataques automatizados.
- ➢ Permite contraseñas por defecto, débiles o bien conocidas, como "Password1" o "admin/admin".
- ➢ Posee procesos débiles o no efectivos para las funcionalidades de olvido de contraseña o recuperación de credenciales, como "respuestas basadas en el conocimiento", las cuales no se pueden implementar de forma segura

Algunas formas de prevenirlo son:

- ➢ Cuando sea posible, implemente la autenticación multi-factor para evitar ataques automatizados de reutilización de credenciales conocidas, fuerza bruta y reúso de credenciales robadas.
- ➢ No incluya o implemente en su software credenciales por defecto, particularmente para usuarios administradores.
- ➢ Implemente un control contra contraseñas débiles, tal como verificar que una nueva contraseña o la utilizada en el cambio de contraseña no esté incluida en la lista de las 10,000 peores contraseñas.

A8 - Fallo de integridad de datos

Los fallos de integridad del software y de los datos están relacionados con código e infraestructura no protegidos contra alteraciones (integridad). Ejemplos de esto son cuando una aplicación depende de plugins, bibliotecas o módulos de fuentes, repositorios o redes de entrega de contenidos (CDN) no confiables

Algunas formas de prevenir este tipo de ataque son:

- ➢ Utilice firmas digitales o mecanismos similares para verificar que el software o datos provienen efectivamente de la fuente esperada y no fueron alterados.
- ➢ Asegúrese que las bibliotecas y dependencias, tales como npm o maven son utilizadas desde repositorios confiables. Si su perfil de riesgo es alto, considere alojarlas en un repositorio interno cuyo contenido ha sido previamente analizado.

Escenario de ataque:

Actualización maliciosa de SolarWinds: Se sabe que los Estados-Naciones utilizan como vector de ataque los mecanismos de actualización, siendo un caso reciente de pública notoriedad el sufrido por

SolarWinds Orion. La compañía que desarrolló el software poseía procesos seguros de construcción y mecanismos de integridad en sus actualizaciones. Sin embargo, estos fueron comprometidos y, durante varios meses, la firma distribuyó una actualización maliciosa a más de 18.000 organizaciones, de las cuales alrededor de un centenar se vieron afectadas. Se trata de una de las brechas de este tipo de mayor alcance y más importantes de la historia.

A9 - Fallas de registro y monitoreo

Volviendo al OWASP Top 10 2021, la intención es apoyar la detección, escalamiento y respuesta ante brechas activas. Sin registros y monitoreo, las brechas no pueden ser detectadas. Registros, detecciones, monitoreo y respuesta activa insuficientes pueden ocurrir en cualquier momento.

Algunas formas de prevenir este tipo de ataque son:

- Asegúrese de que todos los errores de inicio de sesión, de control de acceso y de validación de entradas de datos del lado del servidor se pueden registrar con suficiente contexto como para identificar cuentas sospechosas o maliciosas y mantenerlo durante el tiempo suficiente para permitir un posterior análisis forense.
- Asegúrese de que los registros se generan en un formato fácil de procesar por las herramientas de gestión de registros.
- Asegúrese de que los datos de registros están correctamente codificados para prevenir inyecciones o ataques en el sistema de monitoreo o registros.

Ejemplo:

```
 * Running on http://0.0.0.0:8000/ (Press CTRL+C to quit)
192.168.126.128 - - [31/Aug/2021 19:15:10] "POST /login HTTP/1.1" 200
192.168.126.128 - - [31/Aug/2021 19:15:10] "POST /login HTTP/1.1" 200
192.168.126.128 - - [31/Aug/2021 19:15:10] "POST /login HTTP/1.1" 200
192.168.126.128 - - [31/Aug/2021 19:15:11] "POST /login HTTP/1.1" 200
192.168.126.128 - - [31/Aug/2021 19:15:11] "POST /login HTTP/1.1" 200
192.168.126.128 - - [31/Aug/2021 19:15:11] "POST /login HTTP/1.1" 200
192.168.126.128 - - [31/Aug/2021 19:15:11] "POST /login HTTP/1.1" 200
192.168.126.128 - - [31/Aug/2021 19:15:11] "POST /login HTTP/1.1" 200
```

A10 - Falsificación de Solicitudes del Lado del Servidor (SSRF)

Las fallas de SSRF ocurren cuando una aplicación web está obteniendo un recurso remoto sin validar la URL proporcionada por el usuario. Permite que un atacante coaccione a la aplicación para que envíe una solicitud falsificada a un destino inesperado, incluso cuando está protegido por un firewall, VPN u otro tipo de lista de control de acceso a la red (ACL).

Escenario de ataque:

Exposición de los servicios internos: el atacante puede abusar de los servicios internos para realizar más ataques, como la ejecución remota de código (RCE) o la denegación de servicio (DoS)

ZAP

ZAP es una aplicación para testear la seguridad de una aplicación propia; se le pasa la dirección url y ZAP empieza a probar distintas maneras de atacar y conseguir información de tu aplicación. Después, te pasa un listado de vulnerabilidades encontradas, y varios consejos para resolverlo.

Arquitectura de Software

En el diseño de la arquitectura, tenemos que definir:

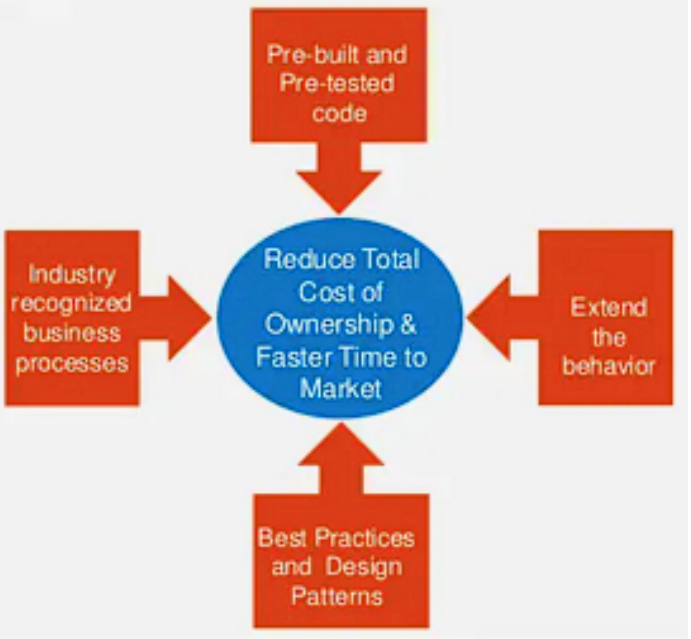
- Nodos y sus configuraciones (diagrama de despliegue)
- Subsistemas y sus interfaces.
- Mecanismos de diseño genéricos:
  - Persistencia (ORM, Concurrencia, Transacciones, ….)
  - UI (web, mobile, desk, responsive,...)
  - Rendimiento (Optimización, Minificación, Cache ….)
  - Distribución (Build, Dependencias, Gestion Configuración, CI/CD, ...)
  - Seguridad (Usuarios, Perfiles, Recursos, …)
  - Monitoreo (Logging, Alarmas,...)
  - Testing (Unidad, E2E, Arquitectura,...)

Frameworks

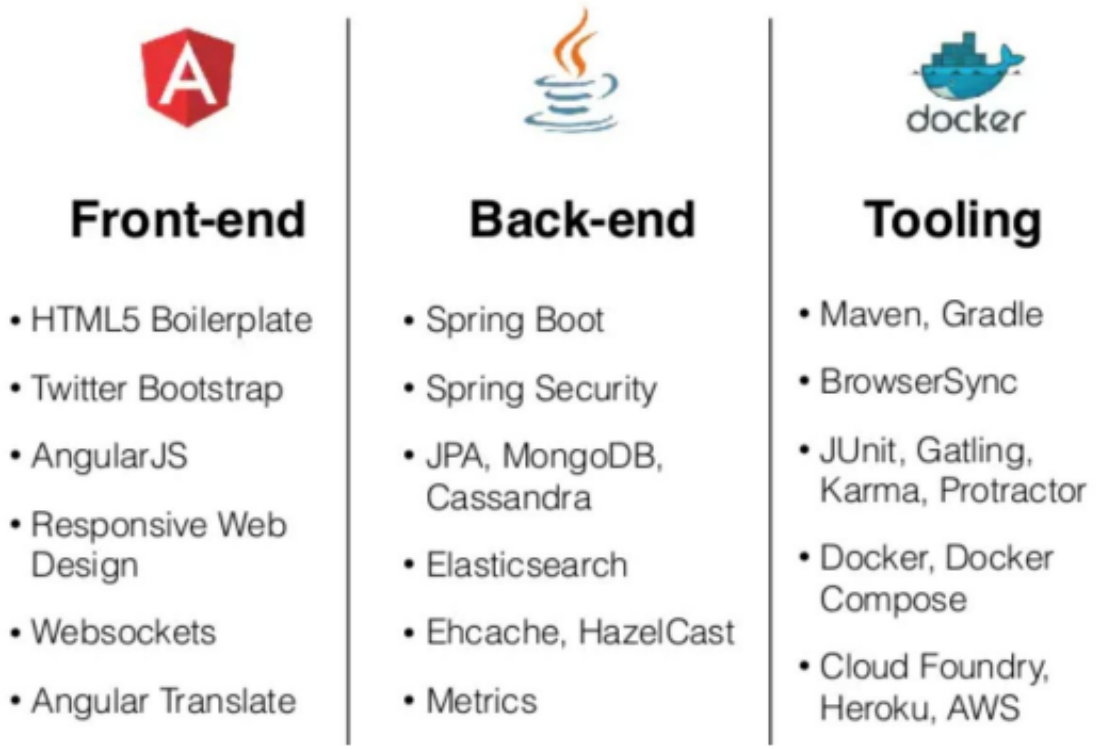
Un Framework es un conjunto de herramientas, librerías y procesos bien definidos y probados, que nos facilitan el desarrollo e implementación de software.

Beneficios:

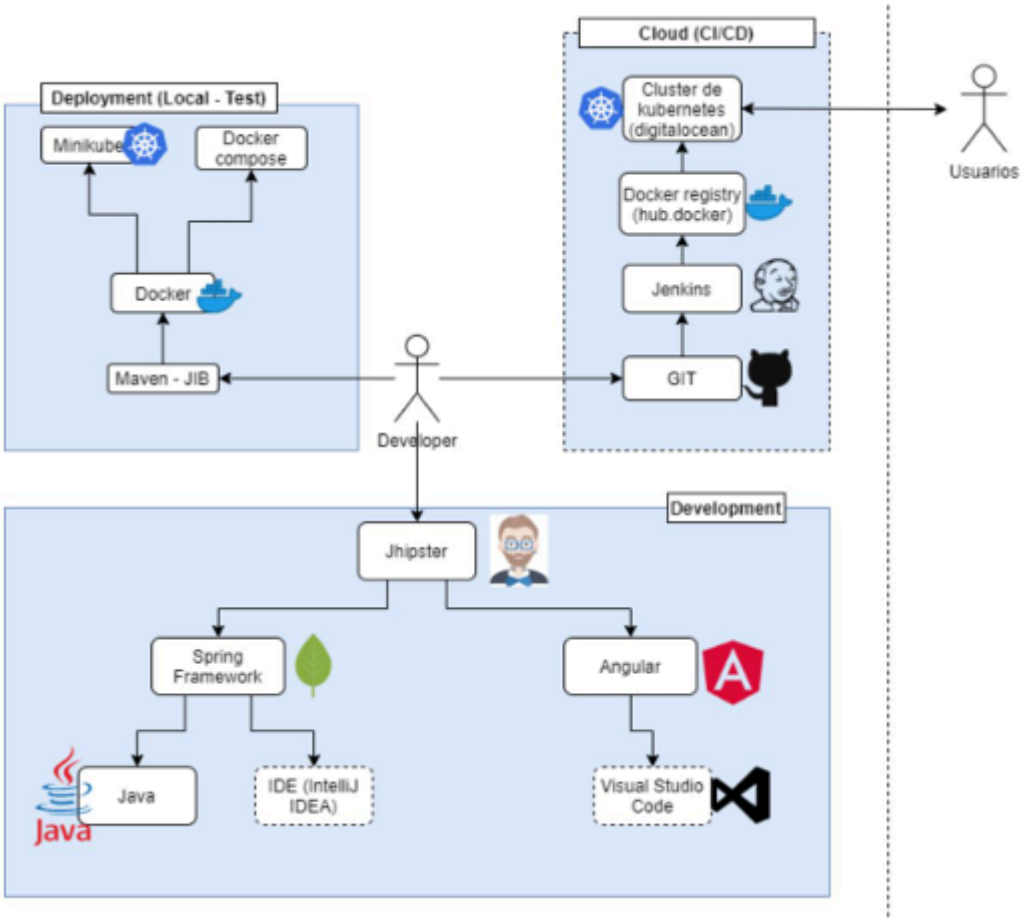
- Reutilización de código previamente testeado.
- Incorpora las mejores prácticas de la industria.
- Permite extender su comportamiento.
- No reinventamos la rueda.



Technology Stack



Framework



JHipster

Para poder lidiar con la complejidad de las arquitecturas y estándares actuales podemos optar por utilizar JHIPSTER que no permite:

"Generar, Desarrollar y Deployar aplicaciones web modernas y arquitecturas de microservicios"

Modelado con un archivo .jdl, que define;

- Entidades
- Relaciones
- Clases a usar (interfaces, tipo de paginado, etc)

A partir del mismo, JHisper

- Crea archivos de configuración
- Crea archivos de proyecto backend (java)
- Crea archivos de proyecto frontend (angular)
- Instala dependencias.

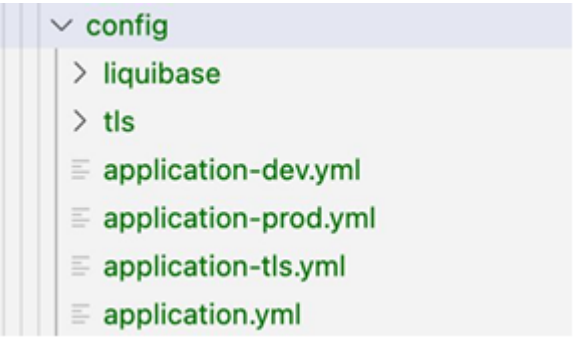
Con la App generada;

- Podemos comprobar que con un modelos de solo 3 entidades se ha generado una infraestructura que implica ***más de 500 archivos***.
- También podemos ver cuales son todas las opciones predeterminadas que utilizó el generador en el archivo ***/.yo-rc.json***, o ejecutando.

  ~/app$ jhipster export-jdl blog-generated.jh

Configuraciones Principales

- Definición de Entornos:
- Definición de Dependencias / Build / Run / Test



Network y DB Config

Tenemos una red compartida por ambos contenedores:

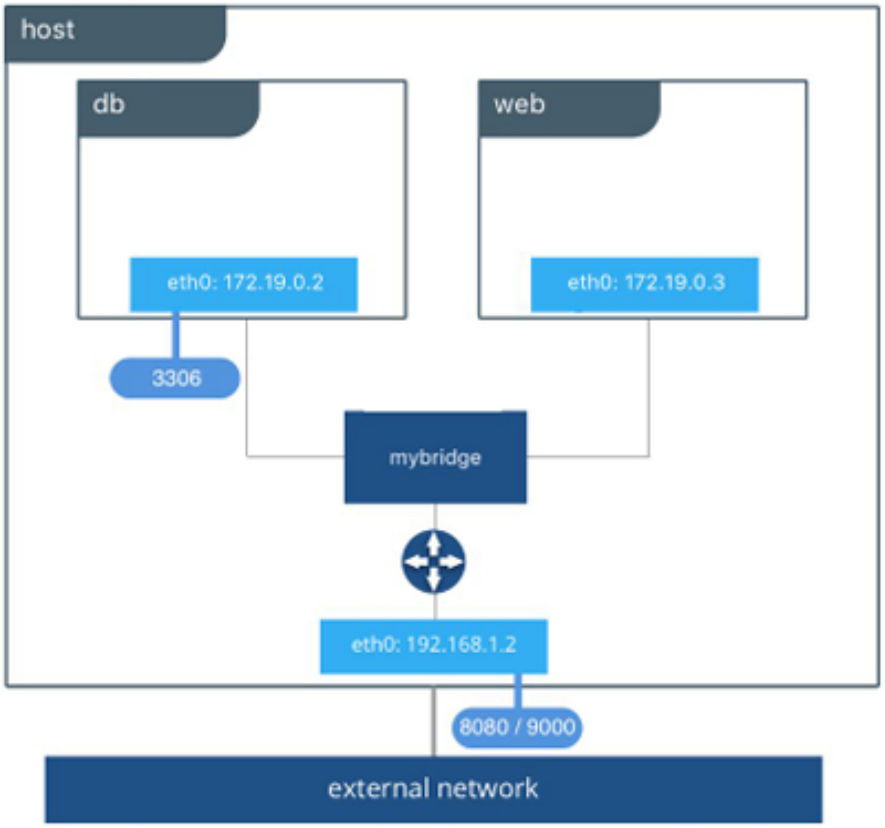
networks:
isa-blog-backend-dev:
driver: 'bridge'

Docker me permite referenciar al contenedor mediante el nombre del servicio:-isa-blog-db

isa-blog-db

Apuntamos en pom.xml y application-dev.yml al nombre del servicio donde está la BD.

<url>jdbc;mariadb://isa-blog-db:3306/blog/</url>



Run y Debug

En una terminal corremos la App utilizando el wrapper de Maven

~/app$ ./mvnw

En otra terminal corremos el frontend mediante el wrapper de npm

~/app$ ./npmw start

Para realizar el debug del backend podemos ejecutar el proyecto desde el Spring Boot Dashboard en VSCode

```
✔ Compiled successfully.
[Browsersync] Proxying: http://localhost:4200
[Browsersync] Access URLs:
 ------------------------------------
       Local: http://localhost:9000
    External: http://172.30.0.3:9000
 ------------------------------------
          UI: http://localhost:3001
 UI External: http://localhost:3001
 ------------------------------------
```

Pruebas Automáticas

La prueba de software es una investigación que permite obtener información acerca de la calidad del producto. Las pruebas reducen riesgos, pero automatizar las pruebas es costoso. Es importante asumir que los errores son parte del proceso. La etapa de pruebas es una inversión, mientras antes y más completo se haga, aumentan las posibilidades de una buena funcionalidad de la app.

Pruebas Unitarias:

- Automatizables: Creada por el desarrollador para ser usada una cantidad ilimitada de veces.
- Completas:
- Repetibles o Reutilizables:
- Independientes:

Pruebas E2E (End to End)

Son pruebas en las cuales yo pruebo la aplicación de Interfaz a Interfaz. No me interesa lo que pasa en medio. Yo pruebo que se pueda llegar de una a la otra.

¿Cómo se comporta en destinos Software? ¿Cómo se siente el usuario con la interfaz? Estas, y otras, son algunas preguntas con las que se plantea y crean este tipo de pruebas.

Pruebas de Integración

Probar todos los elementos unitarios que forman parte de un proceso. Son un conjunto de pruebas para probar todas las características de una funcionalidad en específica.

Pruebas de Sistema

Se quiere probar el sistema en su conjunto, teniendo en cuenta el contexto y el hardware, en un entorno real. Generalmente son de caja negra. Se analizan defectos globales o de comportamiento (seguridad-rendimiento). Ej; probar la independencia, o grado del mismo, de distintas funcionalidades.

Pruebas de validación (Casos de Prueba)

Comprueban que el software producido cumpla con las especificaciones.

Pruebas exploratorias

El desarrollador empieza a probar, de forma desordenada, distintas acciones y secuencias para garantizar la funcionalidad del software. Es importante llevar una agenda del orden de los pasos llevados a cabo, y que estos mismos sean repetibles por otros.

Pruebas de Rendimiento

Se comprueba funcionalidades concretas

- Pruebas de carga
- Pruebas de estrés
- Pruebas de estabilidad (soak testing): Mantener un estrés alto por un periodo largo de tiempo.
- Pruebas de picos (spike testing): Probar “picos” esporádicamente, ver cómo lo maneja, si “suma” esos pico. Ver la sobrecarga al eliminar esos picos, por no eliminar almacenamiento.

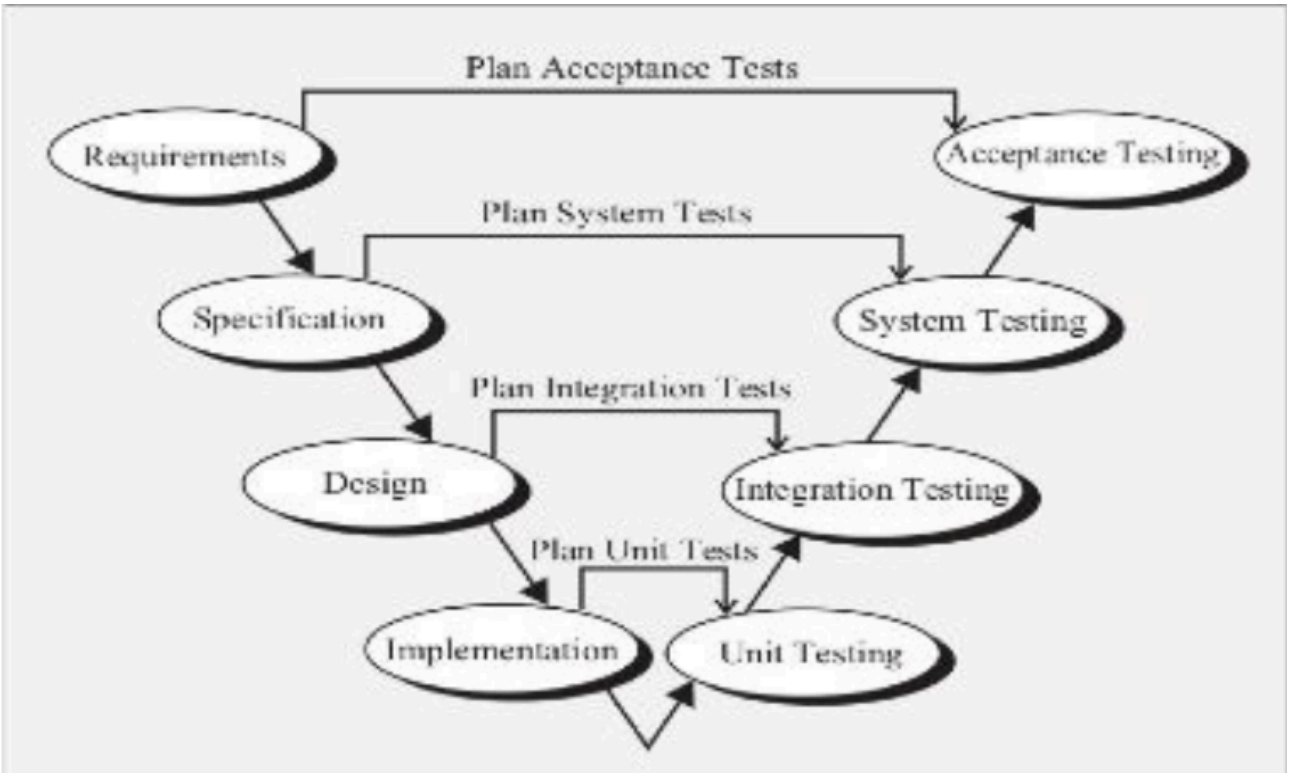
Pruebas de aceptación

Pruebas con las que el cliente aceptará el entregable. Se busca anticipar los pedidos del cliente. Se le pregunta al cliente que va a probar. Y se hacen pruebas para garantizar esa funcionalidad.

Pruebas de Regresión

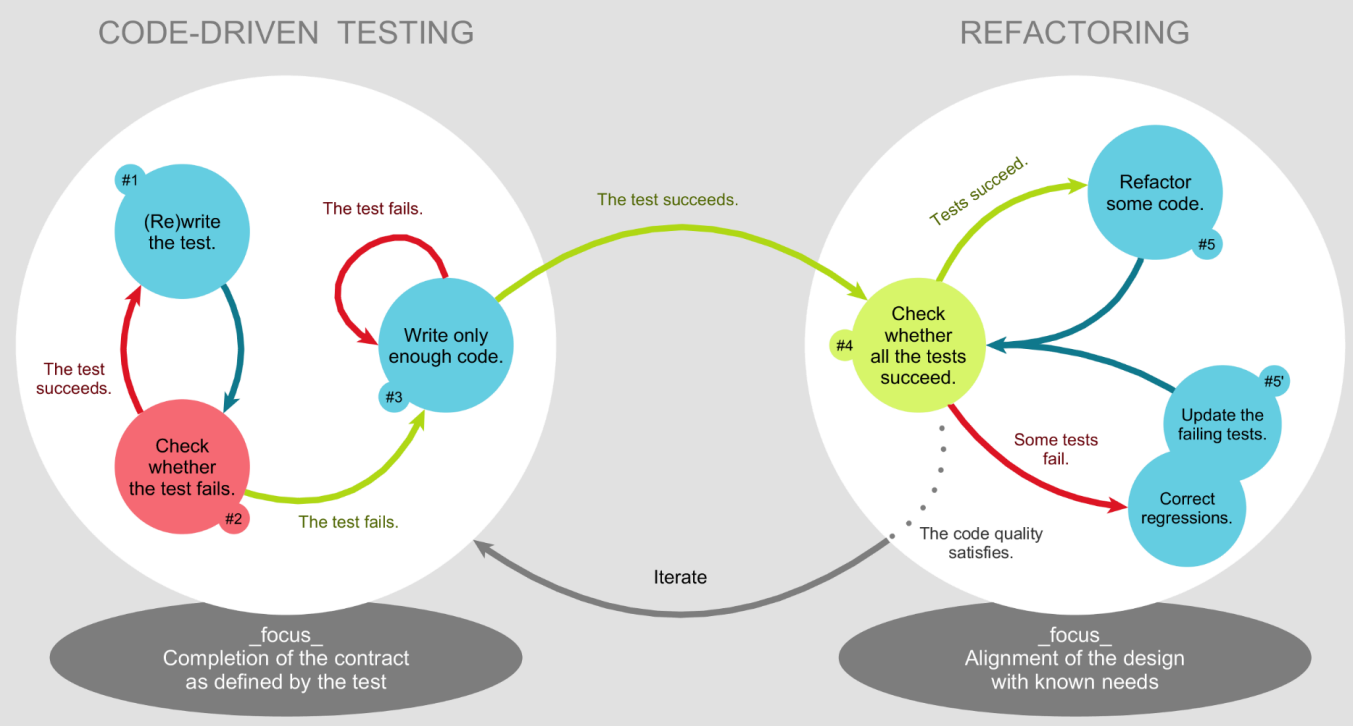
Es cuando agrego a una funcionalidad, tengo que validar el restos del sistema para verificar que no se rompió nada, básicamente.

Modelo desarrollo en V



Test-Driven Development

Es decir, Desarrollo dirigido por tests, TDD por sus siglas en inglés, es una práctica de programación que consiste en escribir primero las pruebas (generalmente unitarias), después escribir el código fuente que pase la prueba satisfactoriamente y, por último, refactorizar el código escrito



La mayor ventaja son los ejemplos: al tener que crear test unitarios antes que la propia función, obliga al desarrollador a pensar ejemplos para verificar la funcionalidad, aumentando la visión del programador. Quizás se le ocurren casos que, de otra forma, no hubiera pensado.

La otra ventaja es que los test unitarios ya quedan hechos.

Automatización de Pruebas

Beneficios:

- Ejecutar pruebas de regresión
- Ejecutar más casos y con mayor frecuencia
- Ejecutar pruebas difíciles de realizar en forma manual
- Lanzar más rápidamente nuevas versiones

Problemas:

- Expectativas poco realistas
- Falsa seguridad (nadie prueba la prueba)
- Costo Mantenimiento: Se ve que no falla, pero no se ve todo lo que costó no funcionara.
- Depender de las herramientas de prueba: Si no hay cierto framework, las pruebas son inejecutables.

TestNG

TestNG es un marco de prueba inspirado en JUnit y NUnit, pero presenta algunas funcionalidades nuevas que lo hacen más potente y fácil de usar.

Ventajas:

- Puede producir informes HTML de ejecución
- Soporta anotaciones
- Los casos de prueba se pueden agrupar y priorizar más fácilmente
- Permite pruebas en paralelo
- Genera registros
- Permite parametrizar los datos

Pasos:

1) Escribir la lógica de negocio de la prueba
2) Insertar las anotaciones de TestNG en el código
3) Agregar la información sobre su prueba (por ejemplo, los nombres de clase, nombres de métodos, nombres de grupos, etc.) en un archivo testng.xml
4) Ejecutar TestNG

Ejemplo de tests:

```kotlin
public class Anotaciones {

      @Test
      public void testCase1() {
            System.out.println("This is the Test Case 1");
      }

      @Test
      public void testCase2() {
            System.out.println("This is the Test Case 2");
      }

      @BeforeMethod
      public void BeforeMethod() {
            System.out.println("This will execute before every
      Method");
      }

      @AfterMethod
      public void AfterMethod() {
            System.out.println("This will execute after every
      Method");
      }
}
```

Grupos de Test:

```kotlin
public class Grupos {

      @Test (groups = { "Car" })
      public void Car1() {
            System.out.println("Batch Car - Test car 1")
      }

      @Test (groups = { "Scooter" })
      public void Scooter1() {
            System.out.println("Batch Car - Test Scooter 1")
      }

      @Test (groups = { "Car" })
      public void Scooter2() {
            System.out.println("Batch Car - Test Scooter 2")
```

```
        }

        @Test (groups = { "Car", "Sedan Car" })
        public void Car1() {
              System.out.println("Batch Sedan Car - Test sedan 1");
        }
}
```

config xml

```XML
<suite name="Suite">
   <test name="Practice Grouping">
       <groups>
           <run>
               <include name="Car" />
               <!--include name="Scooter" /-->
           </run>
       </groups>
       <classes>
           <class name="testNG.Grupos" />
       </classes>
   </test>
</suite>
```

Cypress

Cypress es un framework de testing moderno y todo en uno.

Ventajas

- Una sola dependencia
- Interfaz gráfica para ver el paso a paso
- Espera automática de los elementos
- Grabación automática del estado de la aplicación durante la ejecución de la prueba
- Captura de Video y Pantallas
- Documentación Completa

Navegadores soportados

- Chrome
- Firefox

Buenas prácticas

- Llamar a la api para no repetir código
- Crear entradas en la api para limpiar los datos de prueba o crearlos.
- Crear comandos para no repetir código, ej: logi

Selenium

Selenium es un framework de automatización de navegadores muy usado para pruebas de aplicaciones web, compatible con múltiples lenguajes y navegadores, y sigue siendo uno de los estándares más importantes en QA. Su ecosistema incluye WebDriver, IDE y Grid, lo que permite desde grabar pruebas simples hasta ejecutar suites distribuidas a gran escala.

Componentes principales de Selenium

1) *Selenium WebDriver*: API que permite controlar navegadores de forma programática. Ideal para pruebas robustas, escalables y realistas. Se integra con frameworks como TestNG, JUnit, PyTest, NUnit. Es el componente más usado en entornos profesionales.
2) *Selenium IDE*: Extensión para Chrome, Firefox y Edge.Permite grabar y reproducir interacciones sin programar.Útil para pruebas rápidas, exploratorias o reproducción de bugs.
3) *Selenium Grid*: Permite ejecutar pruebas en paralelo y en múltiples máquinas.Ideal para grandes equipos o pipelines CI/CD.

Ventajas de Selenium

- Multiplataforma: Windows, Linux, macOS.
- Multinavegador: Chrome, Firefox, Edge, Safari.
- Multilenguaje: Java, Python, C#, etc.
- Integración con CI/CD: Jenkins, GitHub Actions, GitLab CI.
- Gran comunidad y documentación.

Limitaciones

- No automatiza aplicaciones de escritorio.
- Requiere más configuración que herramientas modernas como Cypress o Playwright.
- Depende de drivers específicos (ChromeDriver, GeckoDriver).

Gestión de Logs

En informática, se usa el término registro, log o historial de log para referirse a la grabación secuencial en un archivo o en una base de datos de todos los acontecimientos (eventos o acciones) que afectan a un proceso particular (aplicación, actividad de una red informática, etc.). De esta forma constituye una evidencia del comportamiento del sistema.

¿Para qué guardamos los logs?

- Encontrar errores y sus trazas.
- Explicar el comportamiento del sistema en su conjunto.
- Detectar posibles riesgos y amenazas (hackers, fallos repetitivos, etc)
- Conocer el comportamiento de los usuarios
- Obtener datos estadísticos de uso de la aplicación.

Desafíos en el manejo de logs

Generalmente tenemos muchas aplicaciones funcionando en paralelo y reportando sus logs.(servidores, app, sistema). Pasar de un manejo de logs a un análisis de los logs ¿Cómo hacemos para conectar logs de distintas fuentes? ¿Cómo hacemos para referencias y correlacionar esta información

Análisis de logs

- Poder fácilmente conectar distintas fuentes de logs
- Correlacionarse y referenciarlos
- Buscar, filtrar e identificar
- Visualizar lo que está pasando (por ejemplo ctos logs x segundo está generando una determinada aplicación)
- Detección de anomalías y alertas
- ¿Cómo hacemos una retención de los logs sensibles?

ELK

1. Un agente de recogida de logs en nuestras aplicaciones. - *Logstash*.
2. Una base de datos donde almacenar, indexar y buscar los eventos de log de las aplicaciones. - *Elasticsearch*.
3. Una aplicación frontend donde los usuarios puedan consultar los eventos más interesantes y estar informados de cualquier incidencia. - *Kibana*

Componente	***Función principal***	***Tipo***	***¿Por qué es importante?***
Logstash	Ingesta y transformación	Pipeline ETL	Limpia y estructura los logs antes de indexarlos
Elasticsearch	Almacenar, indexar y buscar datos	Motor de búsqueda	Permite consultas rápidas y análisi a escala
Kibana	Visualizar y explorar datos	Interfaz UI	Facilita dashboards, análisis y gestión

¿Qué resuelve Logstash?

- *Falta de consistencia*: puedo estar recolectando logs con distintos formatos y cantidad de información
- *Formato de tiempo*: los formatos no siempre coinciden y el tiempo suele ser fundamental para analizar un proceso

¿Qué resuelve Elasticsearch?

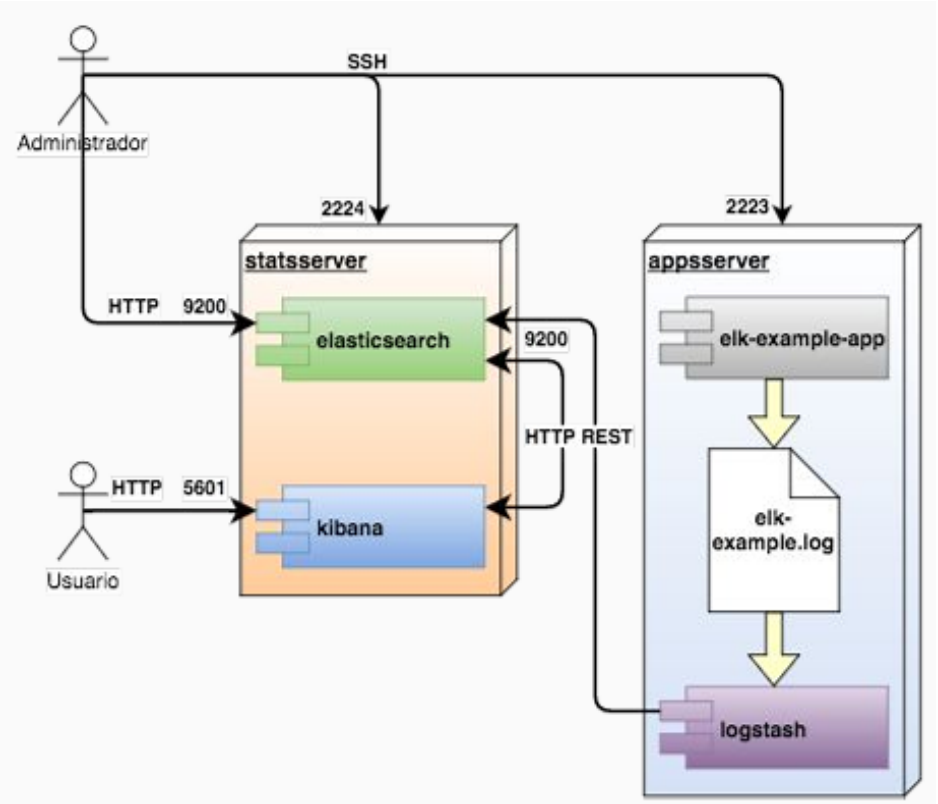
- Necesitás *buscar entre millones* de logs sin esperar minutos.

¿Qué resuelve Kibana?

- Los datos crudos no sirven si no podés interpretar visualmente.

Posible Arquitectura

File Beat capturará los logs de los archivos del sistema y lo enviará a Logstash

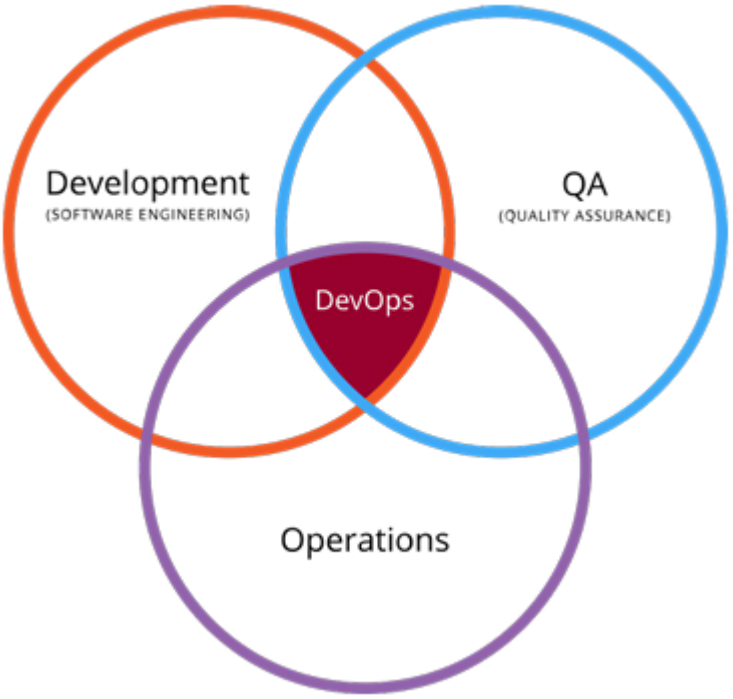


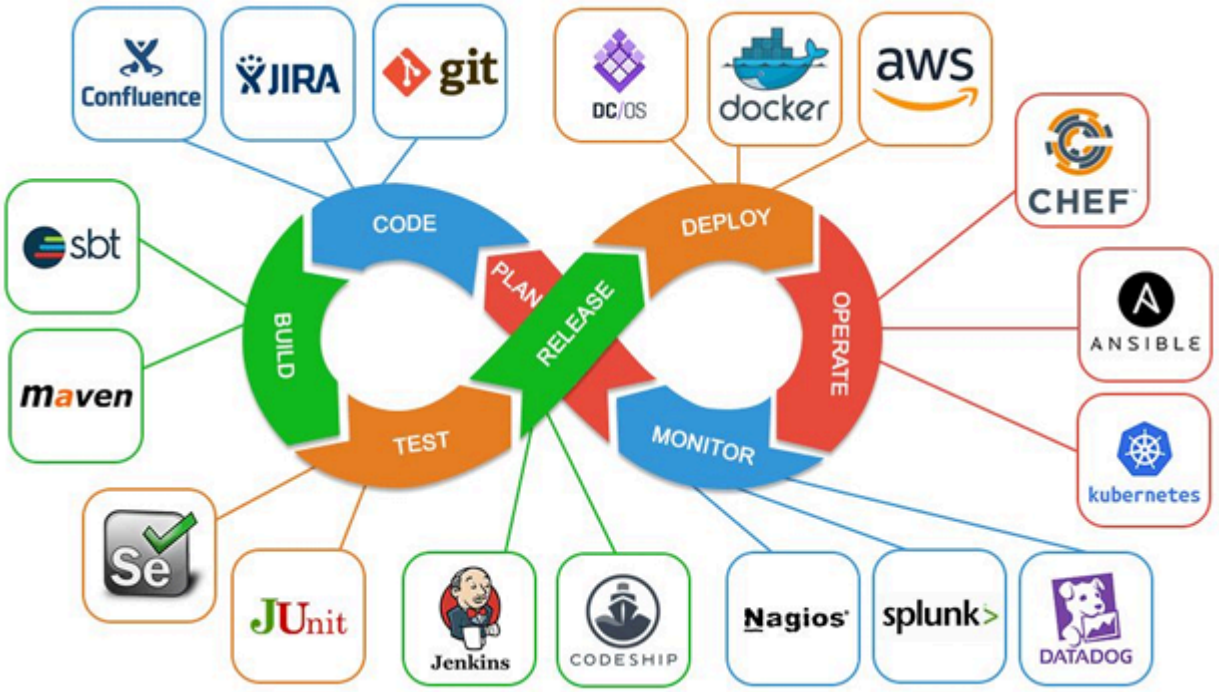
Integración Continua

DEVOPS: Desarrollo + Operaciones

Práctica de Desarrollo

Hay que definir un conjunto de etapas (pipeline), de fases por las que va pasando el software y que serán automatizadas.





¿Qué es la Integración Contínua?

Es una práctica de desarrollo de software mediante la cual los desarrolladores combinan los cambios en el código en un repositorio central de forma periódica, tras lo cual se ejecutan pruebas automáticas y nuevas versiones.

La ***filosofía*** de la integración continua es que; si algo te cuesta, debes hacerlo más a menudo y poco a poco, para que con el tiempo te vaya costando cada vez menos esfuerzo.

Es importante ***Detectar fallos rápidamente***, ya que cuanto más tiempo pasa desde que se introduce un error hasta que se detecta y se resuelve, más nos cuesta solucionar ese error

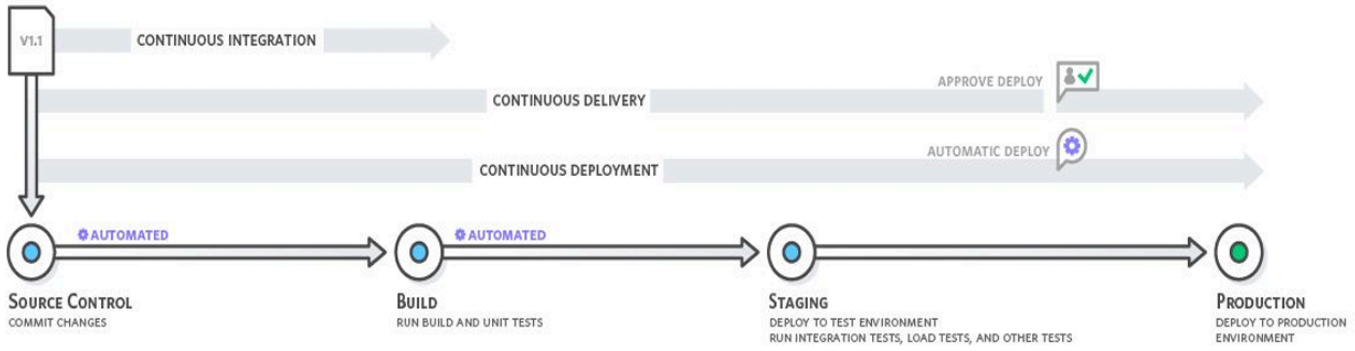
Entrega Contínua: Todo cambio subido al repositorio y que hayan pasado los test son compilados, probados y verificados.

Despliegue Contínuo: Pasar el sistema a producción.

Pipeline: Un ejemplo de un pipeline podría ser que con cada subida de código al repositorio de control de versiones este se descargue y compile.Si está todo correcto, que se ejecuten una serie de pruebas unitarias, o se despliegue el código a otro entorno para hacer pruebas de sistema.

Un pipeline se compone de distintas etapas (stages) secuenciales que ejecutan tareas (steps) que son lanzadas en nodos de trabajo (nodes).

- *Step*: Son las tareas ó comandos que ejecutados de forma secuencial implementan la lógica de nuestro flujo de trabajo
- *Node*: Los nodos son agrupaciones de tareas o steps que comparten un workspace.
- *Workspace*: El directorio de trabajo (workspace) es compartido por los steps del nodo, de forma que steps de un nodo pueden acceder a ficheros/directorios generados por steps de ese mismo nodo.
- *Stage*: Son las etapas lógicas en las que se dividen los flujos de trabajo de Jenkins. Cada stage puede estar compuesto por un conjunto de steps, y cada node puede contener un conjunto de stages



Principios

- Un único repositorio
- Automatizar la compilación del proyecto
- Sistema realiza sus propias pruebas
- Integración diaria (mantener código estable)
- Línea principal operativa
- Reparación de errores inmediata
- Integración rápida (dos fases - una de pruebas)
- Pruebas en una réplica del entorno de producción
- Acceso fácil al último ejecutable
- Buena comunicación (saber estado del proceso)
- Despliegue automático en diferentes entornos

Ventajas	Inconvenientes
Detección temprana de errores	Cambio de procesos habituales
Comunicación constante	Precisa servidores y entornos adicionales
Evita una integración final de gran envergadura	Elaboración de un plan de pruebas propio
Registro exacto de las modificaciones	Puede derivar en demoras cuando varios desarrolladores intentan integrar sus códigos al mismo tiempo
Disponibilidad constante de una versión funcional y actualizada	
Fomenta el trabajo minucioso	

Componentes

Control de versiones: Git, svn

Manejador dependencias: Apache Maven, Apache Ant, scripts shell o programas batch
Software Integración Contínua: Jenkins
Software de reportes: Sonar

Ejemplo: Usar Jenkins File para generar una docker en dockerhub y realizar integracion continua (gracias a JHisper)

PWA - Progressive Web Apps

Las aplicaciones web progresivas (PWA) son aplicaciones web creadas y mejoradas con API modernas para ofrecer capacidades mejoradas, confiabilidad e instalación mientras llegan a cualquier persona, en cualquier lugar, en cualquier dispositivo, todo con un único código base.

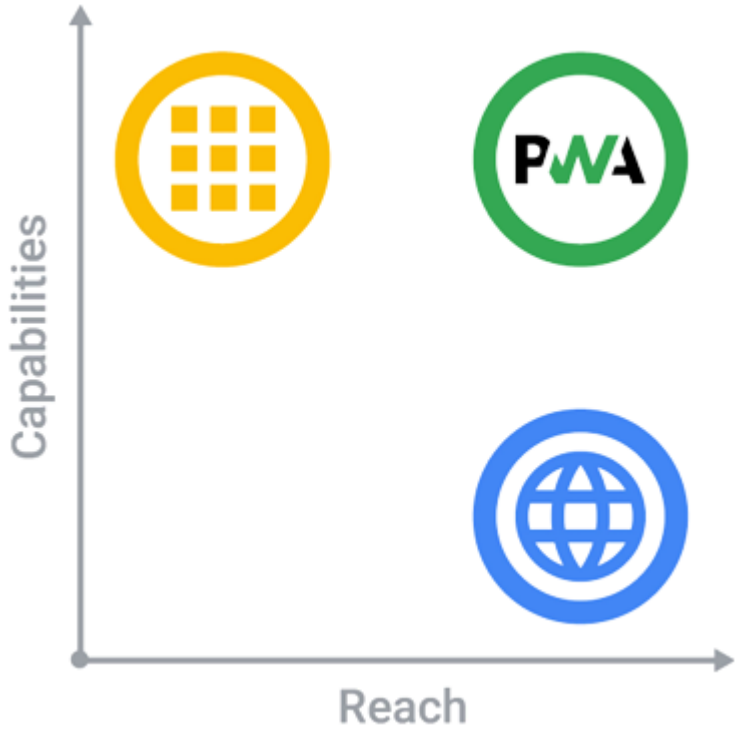
PWA Básica Checklist

- Inicia rápido, permanece rápido
- Funciona en cualquier navegador
- Se adapta a cualquier tamaño de pantalla
- Proporciona una página sin conexión
- Es Instalable

PWA Optima Checklist

- Provee de una experiencia sin conexión
- Es completamente accesible
- Utiliza capacidades poderosas cuando están disponibles
- Se pueden descubrir mediante búsquedas
- Funciona con cualquier tipo de entrada
- Proporciona contexto para las solicitudes de permisos
- Sigue las mejores prácticas de código saludable

Capacidad y Alcance



Estrategias para comenzar

- Hacer la aplicación instalable
- Enfocarse en funcionalidades
- Construir una versión simple
- Comenzar desde cero

Sitios Blandos

¿Qué papel juega el navegador a la hora de determinar la experiencia de usuario del sitio?

El control que los diseñadores conocen en el medio impreso, y que a menudo desean en el medio web, es simplemente una función de las limitaciones de la página impresa. Debemos aceptar que la web no tiene esas mismas restricciones y diseñar para esta flexibilidad. Pero primero debemos aceptar el flujo y reflujo de las cosas.

Ingredientes del Diseño Responsive

- Grillas fluidas
- Medio flexible
- Consultas de medios

Ahora más que nunca, estamos diseñando trabajos destinados a ser vistos a lo largo de un gradiente de diferentes experiencias. El diseño web responsivo nos ofrece un camino hacia adelante, permitiéndonos finalmente 'diseñar para el flujo y reflujo de las cosas'

El Mito del Usuario Móvil:

- 4889 resoluciones de pantalla únicas
- 53% de los usuarios representados por las 10 resoluciones más comunes
- 100% de uso de pantalla táctil entre las 30 resoluciones principales cuando se considera el tipo de entrada

Todo va Primero

*Cuando un equipo diseña **Mobile First**, el resultado final es una experiencia enfocada en las tareas clave que los usuarios quieren realizar, sin desvíos innecesarios ni basura de interfaz que llena los sitios web de escritorio actuales.Eso es buena experiencia de usuario y bueno para el negocio.*

- ➢ Identificar la funcionalidad central.
- ➢ Hacer que esa funcionalidad esté disponible usando la tecnología más simple posible.
- ➢ ¡Mejorar!

Feature Detection

✓ CSS @supports
- ✓ Detección de características con 'feature' in object.style
- ✓ module/nomodule
- ✓ Cutting the Mustard
- ✗ User-Agent Sniffing

Adoptar una perspectiva de rayos X significa mirar 'a través' de los widgets complejos y los estilos visuales de un diseño, identificar el contenido central y las piezas funcionales que componen la página, y encontrar un equivalente simple en HTML que funcione universalmente.

Diseño Intrínseco

Puedes realmente hacer que el diseño sea intrínseco al contenido que tenemos y al diseño que queremos hacer

El Diseño Web es el Diseño de las Restricciones

Construyendo una Fundación Sólida

Principios:

- Usar mobile para focalizar
- Concéntrate en él contenida y la funcionalidad central
- Mejora progresivamente

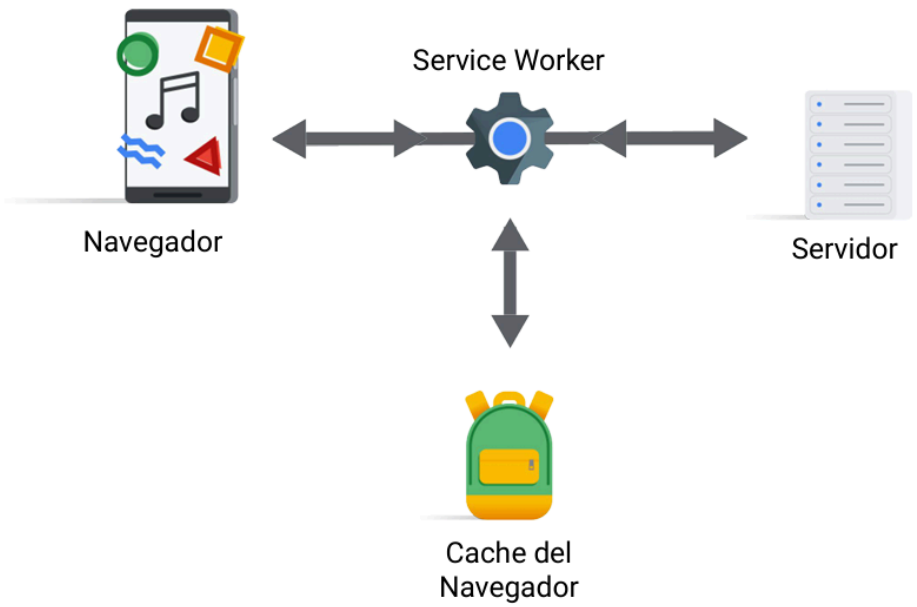
Trabajando Sin Conexión

¿Cómo podría un proxy en el navegador ayudar a que su PWA funcione sin conexión?

Los jugadores clave

- ❖ Service Workers
  - ➢ Un proxy en el navegador
  - ➢ Funciona en línea o desconectado
- ❖ Cache Storage API
  - ➢ Administra la caché del sitio
  - ➢ Agrega/elimina/verifica elementos de caché

Ciclo de vida de una Solicitud al Proxy



Service Worker Scope

URL	***Estado***	***Razón***
https://example.com/app	Yes	“Coincide con la raíz del scope”
https://example.com/app/css/style.css	Yes	“Está por debajo de la raíz del scope”
https://example.com/other-app	No	“Está por encima de la raíz del scope”
https://example.com/js/shared.js	No	“Está por encima de la raíz del scope”

Service Worker Update

URL	***¿Contenido cambió?***	***¿Actualiza SW?***	***Razón***
https://example.com/app/sw.js	No	No	“No cambió el contenido, misma r del service worker”
https://example.com/app/sw.js	Yes	Yes	“Cambió el contenido, misma ruta service worker”
https://example.com/app/sw.12345.js	Yes	No	“Ruta del service worker diferente
https://example.com/js/sw.js	No	No	“Ruta del service worker diferente
https://example.com/js/sw.js	Yes	No	“Ruta del service worker diferente

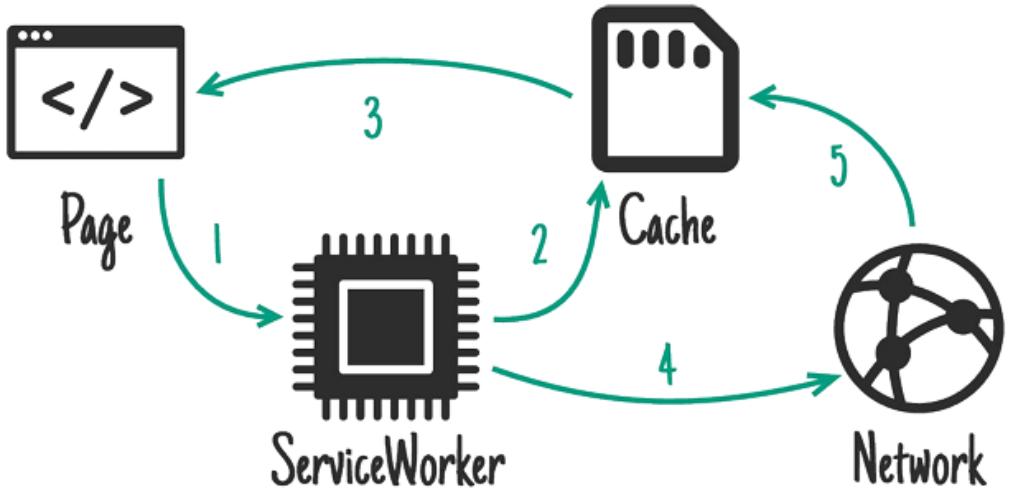
Estrategias de Caché

¿Podría almacenar en caché el contenido que se actualiza con frecuencia?

Obsoleto-Mientras-Reválida

Esta estrategia devuelve inmediatamente la versión en caché (stale), asegurando una carga instantánea, mientras en segundo plano realiza una solicitud a la red para obtener una versión actualizada (revalidate). Cuando la nueva versión llega, se guarda en la caché para la próxima visita. Es una de las estrategias más equilibradas porque combina velocidad y frescura.

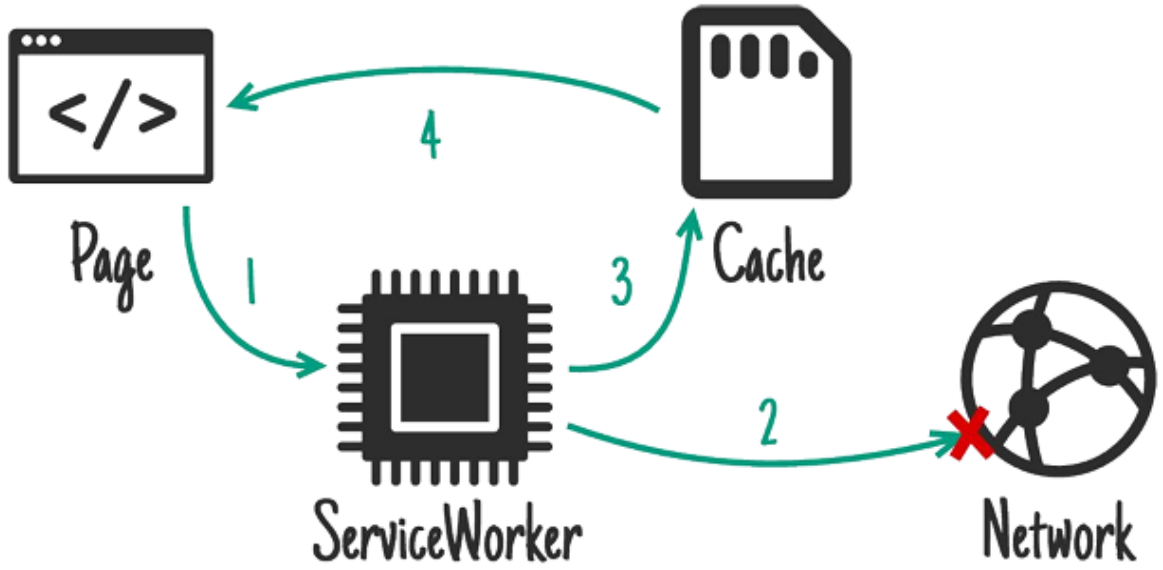
Es especialmente útil para recursos que cambian, pero no de forma crítica, como imágenes de productos, listas de artículos o contenido semidinámico. El usuario obtiene una experiencia rápida sin sacrificar la actualización eventual. Su desventaja es que el usuario puede ver contenido viejo por unos segundos, pero suele ser un compromiso aceptable.



Primero la red

En Network First, el service worker intenta obtener la respuesta desde la red primero. Si la red responde rápido, se devuelve la versión más reciente y se actualiza la caché. Si la red falla o es demasiado lenta, se usa la versión en caché como respaldo. Esta estrategia es ideal para contenido dinámico que cambia frecuentemente, como noticias, timelines, dashboards o datos de APIs.

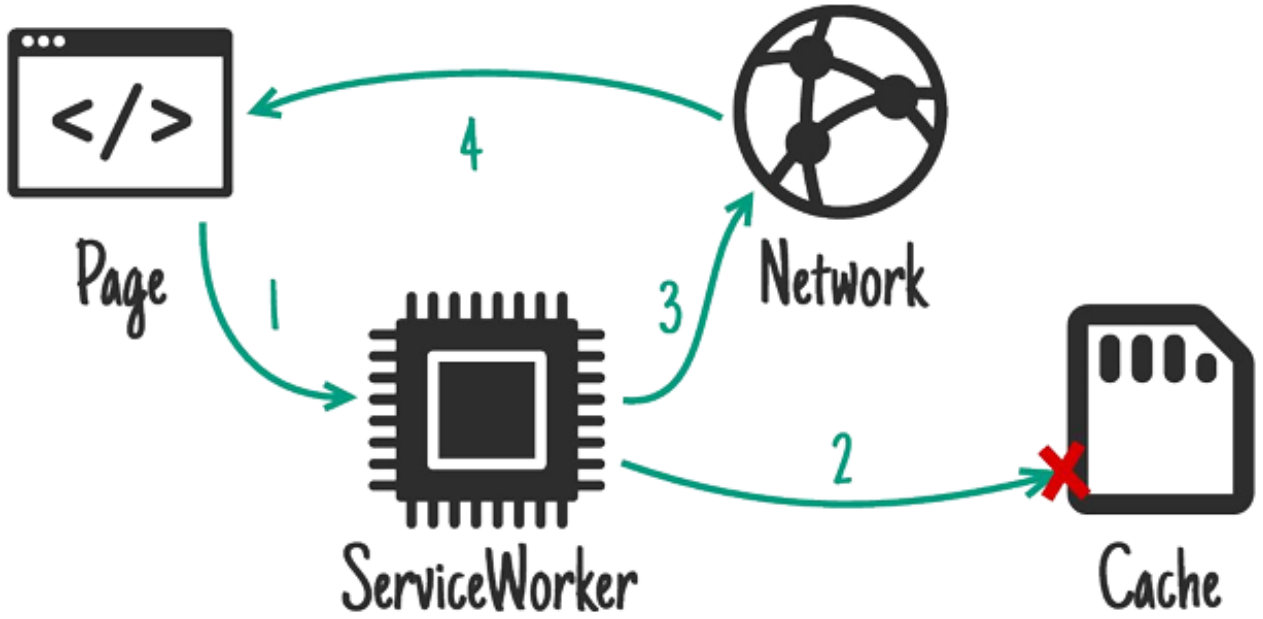
Su ventaja es que prioriza la frescura del contenido, asegurando que el usuario vea la información más actual posible. La desventaja es que, si la red es lenta, la experiencia puede degradarse. Por eso se recomienda combinarla con timeouts o fallback offline para evitar bloqueos prolongados.



Cache Primero

La estrategia Cache First intenta siempre responder desde la caché antes de ir a la red. Si el recurso está almacenado localmente, se devuelve de inmediato, lo que hace que la aplicación cargue extremadamente rápido y funcione incluso sin conexión. Solo si el recurso no está en caché, el service worker realiza una solicitud a la red y luego guarda la respuesta para futuras visitas. Es ideal para recursos estáticos que cambian poco, como imágenes, fuentes, íconos o el app shell.

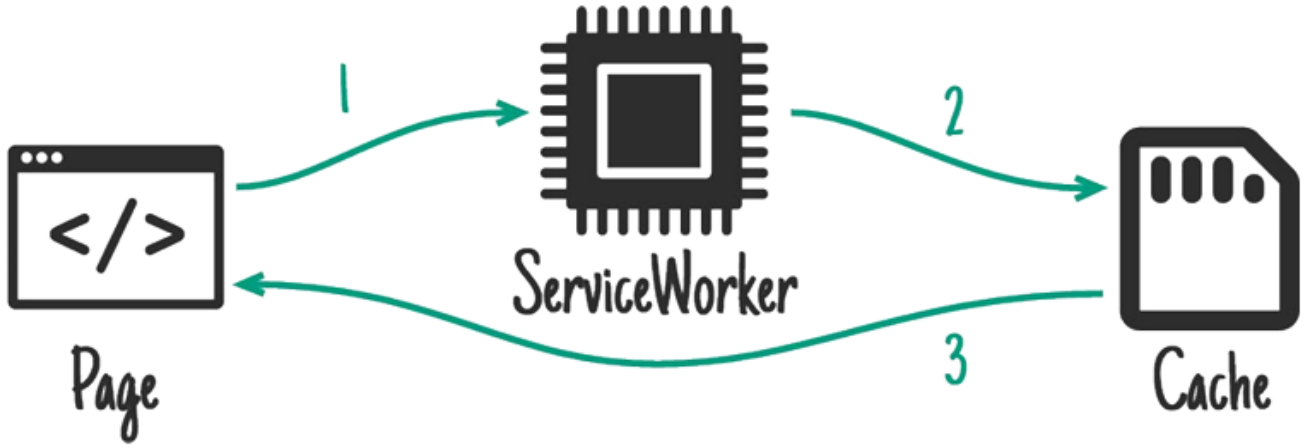
Su principal ventaja es la velocidad y la confiabilidad, especialmente en contextos de mala conectividad. Sin embargo, su desventaja es que puede servir contenido desactualizado si no se implementa un mecanismo de actualización (como versionado de caché o stale-while-revalidate). Por eso se usa cuando la frescura no es crítica, pero la disponibilidad sí lo es.



Cache Only

Esta estrategia responde exclusivamente desde la caché y nunca consulta la red. Es extremadamente rápida, pero solo funciona si todos los recursos necesarios ya están precargados. Se usa en casos muy controlados, como aplicaciones totalmente offline, demos, kioscos o recursos que nunca cambian.

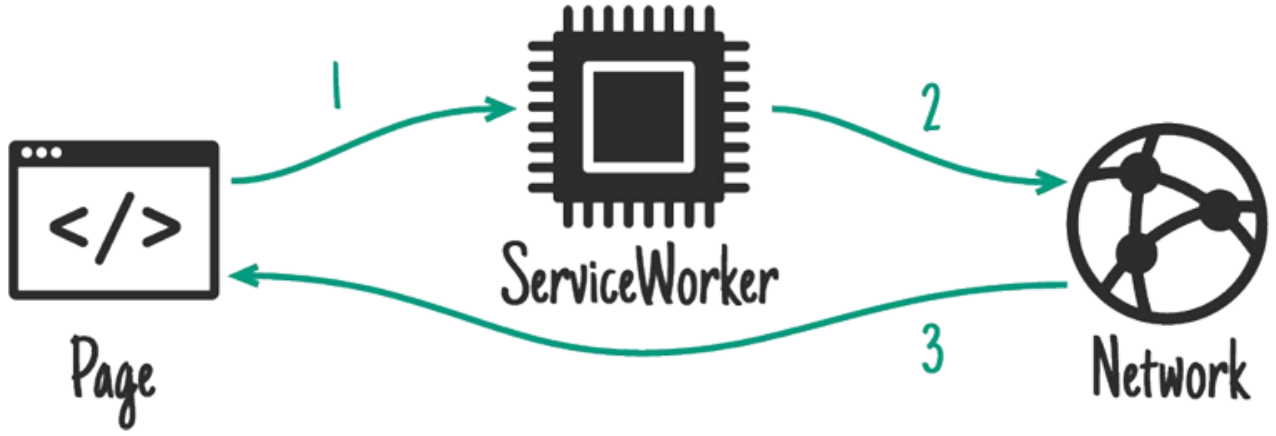
Su ventaja es la velocidad y la predictibilidad. Su desventaja es que no puede actualizarse ni recuperar recursos nuevos, por lo que es una estrategia muy limitada y rara vez usada en producción



Network Only

En Network Only, todas las solicitudes van directamente a la red y nunca se usa la caché. Es la estrategia más simple y la menos usada en PWAs, porque elimina la capacidad offline. Sin embargo, es útil para endpoints que no deben ser cacheados por razones de seguridad o integridad, como operaciones de login, pagos, o solicitudes POST.

Su ventaja es que garantiza que el contenido siempre sea el más reciente y evita inconsistencias. Su desventaja es obvia: si no hay conexión, no hay respuesta. Por eso se usa solo en casos muy específicos donde la caché sería peligrosa o incorrecta.



WorkBox

Workbox es un conjunto de librerías creadas por Google para simplificar el manejo de service workers y estrategias de caché en PWAs. Mantener manualmente la lógica de caché, actualización, expiración y rutas puede volverse complejo a medida que una app crece. Workbox abstrae estas tareas y ofrece módulos especializados para cada necesidad.

En lugar de escribir a mano listeners de ***fetch***, ***install*** y ***activate***, Workbox te permite declarar rutas y asignarles estrategias de caché con una sintaxis clara. También incluye herramientas para precaching, background sync, expiración, routing avanzado y generación automática del service worker durante el build.

Cómo decide qué estrategia usar

El módulo ***workbox-routing*** permite interceptar solicitudes y decidir qué estrategia aplicar según patrones de URL, tipo de recurso o reglas personalizadas.

Ejemplo conceptual:

- ➢ Todo lo que sea /api/* → Network First
- ➢ Todo lo que sea /images/* → Stale-While-Revalidate
- ➢ Todo lo que sea .css → Cache First

https://workshops.page.link/pwa03--workbox

IndexedDB

¿En qué crees que se diferencia IndexedDB del almacenamiento en caché?

Cache Storage API

✓ Recursos de Red

✓ Contenidos basado en archivos

IndexedDB

✓ Información estructurada

✓ Datos donde buscar